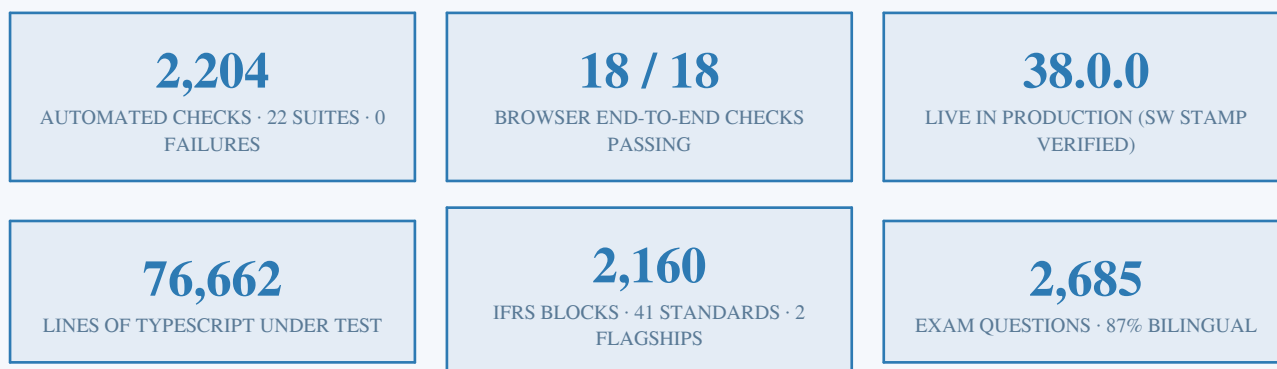


Table of Contents

Executive Summary	2
1. Verification Campaign - Every Layer, Every Result	3
1.1 Method: six independent layers	3
1.2 Static gates and the debt scan	3
1.3 The v38 database guard passes its first real test	3
1.4 End-to-end: 18/18 in a live browser	5
1.5 Production probes: the deployment is current	6
2. The v38 Release Under the Microscope	7
2.1 Five recommendations, one release	7
2.2 The resilience layer: guard and backoff	7
2.3 The fieldwork features: print and bridge	7
2.4 IFRS 16 at the flagship bar	8
2.5 What changed in the numbers	8
3. Product Inventory - What Exists Today	9
3.1 The platform at a glance	9
3.2 The learning spine	9
3.3 Exam preparation	9
3.4 The reference library	9
3.5 AI tooling	10
4. Health Assessment - Where We Stand	11
4.1 Strengths	11
4.2 Risk register	11
4.3 The CI blind spot	12
5. What We Can Do - The Roadmap	13
5.1 Immediate: repair the verification machinery	13
5.2 The next one to two releases	13
5.3 The strategic horizon	14
5.4 Recommended scope for v39	14
6. Release History and Reproducing the Audit	15
6.1 Version timeline v20 to v38	15
6.2 Reproducing this audit	15

Executive Summary

Two questions drove this campaign: after shipping v38 - the release that implemented all five recommendations of the previous audit - **where does the product actually stand**, and **what should the next releases do with that position?** To answer them honestly, the entire verification architecture of the project was re-run from scratch rather than trusting the release notes: static analysis, the database environment guard, the full regression battery, a live browser end-to-end walkthrough, production probes against the deployed build, and a fresh census of every content asset. Nothing was taken on faith, and one thing the campaign found was not in any release note at all.



The headline finding is positive with one asterisk. Every layer that can be run locally or probed remotely is green: the type checker and linter are clean, the auto-restore database guard that v38 introduced works exactly as designed (it passed both its standalone probe and its place in the e2e boot path), the battery grew from 21 suites and 2,117 checks to 22 suites and 2,204 checks with zero failures, all 18 browser end-to-end checks pass against a real running instance, and the production deployment is verifiably current - the service worker stamps auditedge-v38, the version constant 38.0.0 is present in the served JavaScript chunks, and the v38 feature strings (the ICQ print engine, the program bridge, the IFRS 16 rewrite) are all in the code the live site ships to users. The asterisk: **the project's own CI pipeline has failed 23 consecutive times** - every push since 27 September - and nobody noticed, because deployments go through Vercel and never touch CI. The failure is not a code defect: the CI job runs the battery against an empty scratch database, tripping the exact 15 zero-count failures the v37 audit diagnosed and v38's guard fixed locally - but the workflow was never updated to call the guard. The two most recent runs fail differently still: GitHub never assigned them a hosted runner, and after fifteen minutes of waiting they were cancelled. The product is verified; the verification itself is not yet automated end to end. That is the single highest-value fix available, and it costs a few lines of YAML.

The roadmap that follows from this state is unusually clear. Immediately: repair CI by wiring the existing database guard into the workflow, fix its corrupted branch filter, pin the runner image, and make the check required - turning the 2,204-check moat into an always-on gate instead of a manual ritual. Next release: a third IFRS flagship to the 98-block depth bar (IFRS 9 is the natural candidate by exam weight), the print engine generalized beyond the questionnaire to audit programs and findings lists, and a guided engagement flow that connects the pieces v37 and v38 built into one continuous fieldwork journey. The strategic horizon - multi-user workspaces, durable production data, offline-first fieldwork - is about who the product serves rather than what it

contains, and each item now has a concrete starting point in the codebase.

1. Verification Campaign - Every Layer, Every Result

1.1 Method: six independent layers

The campaign applied the same discipline as the v37 audit, with one layer added. **Layer one** is static: the TypeScript compiler and ESLint over the whole tree. **Layer two** is environmental: the v38 database guard, exercised both as a standalone command and from inside the e2e boot sequence, because a guard that only works in one path is half a guard. **Layer three** is the regression battery - twenty-two suites chained into a single command. **Layer four** is end-to-end: a real browser driving the running application through the newest features. **Layer five** is production: HTTP probes, service-worker inspection, and chunk-level string verification against the deployed build. The layer the v37 audit performed manually and this one formalizes is the sixth: repository infrastructure - the state of CI, which turned out to hold the campaign's only red finding.

1.2 Static gates and the debt scan

Both gates pass clean on the v38 tree. `tsc --noEmit` reports no errors across the 76,662-line source tree, and ESLint exits zero with no warnings. A supplementary debt scan found **zero TODO, FIXME, HACK or XXX markers** anywhere in `src/` - the codebase carries no acknowledged technical debt in the conventional marker sense, which is unusual for a project of this age and is worth stating plainly: debt here lives in architecture decisions (documented in the risk register of chapter 4), not in flagged code smells. The repository itself is disciplined too - a clean working tree, thirty-three commits on main, and the v38 release commit (bbe58a8) pushed and confirmed identical on origin.

1.3 The v38 database guard passes its first real test

The v37 audit opened with an embarrassing discovery: its own first battery run failed fifteen checks because a fresh sandbox ships an empty 274 KB database skeleton while the real content lives in a 19.6 MB gzipped snapshot that had to be restored by hand. v38 shipped the fix - `scripts/ensure-db.ts`, a guard that probes the database (a size fast-path, then bank and course counts) and restores the snapshot automatically when things look thin, wired into predev and the dev boot scripts. This campaign verified the guard in three states. Against the healthy 20.5 MB database it fast-pathed in milliseconds, reporting `bank=2685 courses=41` and touching nothing. Inside the e2e boot path it ran green before the server started. And the battery that followed it passed every database-dependent check - the failure mode that produced fifteen red lines a version ago now cannot occur on any machine that boots the project normally. The irony that CI still suffers the exact failure this guard eliminates is examined in chapter 4.

Suite	Checks	Outcome
sectors-v15 · audit program library	515	0 failures
v23 · ACCA deep past papers	382	0 failures

Suite	Checks	Outcome
v22 · past-paper question bank	366	0 failures
v38 · resilience, IFRS 16, print, bridge	87	0 failures
v37 · Test of Control	75	0 failures
v24 · CFA/CMA/CPA skills bank	70	0 failures
v27 · podcast courses	68	0 failures
v30 · deep links + exam archive	67	0 failures
v26 · flagship pools	66	0 failures
v32 · IFRS all-41 depth	57	0 failures
v25 · question generator	51	0 failures
v34 · DipIFR past papers	48	0 failures
v20 · exam engine + SRS + ISA spine	47	0 failures
v36 · IFRS 15 flagship depth	45	0 failures
v31 · IFRS Summaries	44	0 failures
v35 · in-app exam viewer	43	0 failures
v21 · IFRS 18 + Egypt Arabic parity	32	0 failures
v28 · edge TTS voices	31	0 failures
models-v15 · AI registry + live routing	31	0 failures
engagement-v12	28	0 failures
analyzer-v11 · trial balance	27	0 failures
v29 · engagement smoke	24	0 failures
Total - 22 suites	2,204	0 failures

Table 1 - Regression battery results per suite (run of 5-6 October 2026; database verified healthy by the ensure-db guard before the run).

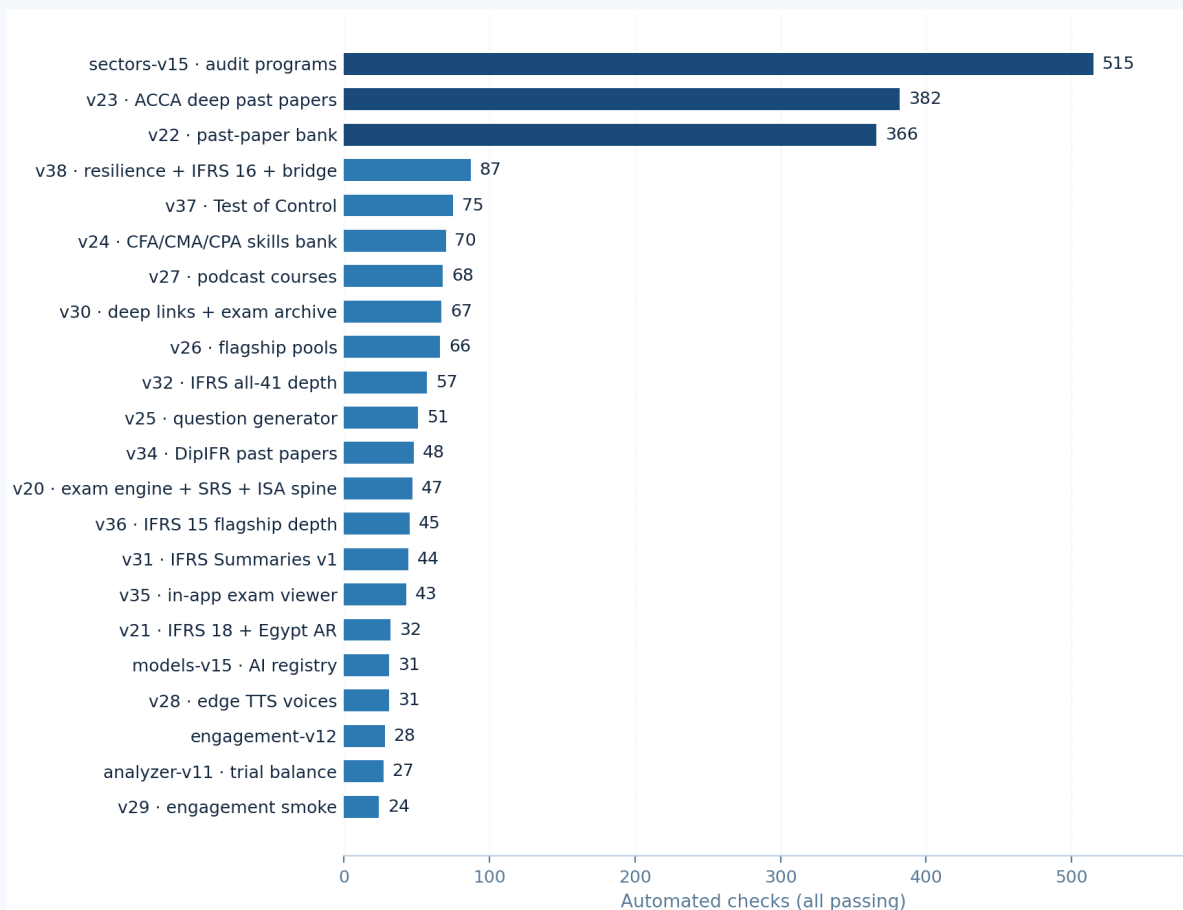


Figure 1 - Regression battery: 2,204 automated checks across 22 suites, zero failures. The v38 suite (87 checks) is the release's own proof.

Two details make this run more meaningful than the raw total. First, the arithmetic of growth is exactly accounted for: v37 closed at 2,117 checks across 21 suites, v38 adds its own 87-check suite, and the battery lands at 2,204 across 22 - every prior suite carried forward unmodified and still green, which is the real test of a regression architecture (old locks must keep working when new ones are cut). Second, the models suite performs **live AI calls** against the keyless pool as part of the battery - and this run completed without a single 429, with the workspace engine serving GLM answers and the vision check routing through the ovh-vision engine. The previous audit watched that suite pass while absorbing live rate-limit errors; the v38 backoff layer plus a healthier pool made this run clean, which is circumstantial but encouraging evidence that the resilience work is doing its job.

1.4 End-to-end: 18/18 in a live browser

The browser suite (scripts/e2e-v38.sh) boots the real application and drives it through the release's flagship paths with a headless agent browser. All eighteen checks passed:

Boot integrity. The ensure-db guard ran green inside the boot sequence, the server came up, and the sidebar displays the v38 release badge.

The questionnaire engine. A banking Test-of-Control questionnaire rendered with both core and industry module sections intact.

Blank ICQ print. The new print action built an A4 fieldwork copy with 102 tick-boxes (three per question), a notes column, and the fieldwork instruction note.

Verdict print. After answering, the run produced a STRONG verdict at 100%, and the report print renders the verdict banner, the weighted score, and the answers marked directly on the boxes.

The bridge. From the verdict view, the bridge button navigated to the audit program view, the customizer opened automatically, the industry was pre-filled from the questionnaire, and the concerns carried the live ICQ verdict context - the intended questionnaire-to-program handoff, verified end to end.

Arabic parity. The app hydrated fully right-to-left in Arabic, the Arabic Academy family renders with all 22 courses, 402 SVG covers painted in the main area, and the navigation shell localizes completely.

1.5 Production probes: the deployment is current

The deployment at `auditedge-snowy.vercel.app` was probed at the HTTP and chunk level to confirm the live site is the v38 build and not a stale artifact:

Probe	Result
Root URL	HTTP 200 in 39 ms (15.9 KB shell)
Service worker cache stamp	auditedge-v38 (current)
Version constant in served chunks	38.0.0 present
v38 feature strings in chunks	ICQ print, program bridge, IFRS 16 all found
ToC generate API guard	400 on invalid body (correct)
Bank API	200 - 2,685 questions, 2,330 with Arabic
Bank area split	accounting 1,668 · auditing 640 · ethics 274 · Egypt 103
Bootstrap API	HTTP 200
Web manifest	200, application/manifest+json
Exam paper sample (2024-12)	200, application/pdf, 966 KB
CI pipeline (GitHub Actions)	FAILED - 23 consecutive runs (see ch. 4)

Table 2 - Live production probes against `auditedge-snowy.vercel.app`, 6 October 2026. The production build was also compiled locally: exit 0.

2. The v38 Release Under the Microscope

2.1 Five recommendations, one release

The previous audit closed with five recommendations. v38 shipped all five in one release - an unusually complete response that deserves its own examination rather than a bullet point. This chapter takes each recommendation, locates it in the code, and asks whether the implementation actually delivers the intent.

Recommendation	Shipped as	Verified by
1. Auto-restore DB guard	scripts/ensure-db.ts - size fast-path, bank/course probe, snapshot restore + verify; wired as predev, ensure:db and inside dev-clean.sh	Standalone probe (fast-path on healthy DB), e2e boot path, battery
2. ICQ PDF export	src/lib/toc/icq-print.ts - hidden-iframe print engine, CSS tick-boxes, full RTL, HTML-escaped AI text; two actions in the ToC runner	e2e: blank copy with 102 boxes + verdict report print
3. AI retry with backoff	src/lib/backoff.ts (pure helpers: Retry-After parser, exponential backoff with jitter) + per-engine cooldown map in the keyless pool + keyed-path backoff in ai.ts	Unit-covered by the battery; models suite ran clean with zero 429s
4. IFRS 16 flagship rewrite	63 to 98 blocks, flagship: true - one running case (Delta Co) through the whole lessee engine, lessor mirror, sale-and-leaseback, remeasurement journals with amounts	measure-ifrs16.ts depth numbers + strings in live production chunks
5. ToC to program bridge + Arabic verification	programTailorPrefill in the store, seeded AiTailorDialog, effect-free forced-open pattern; verify-ar-courses.ts for the 22 Arabic Academy courses	e2e: bridge navigation, auto-open, prefill, verdict context, RTL pass

Table 3 - The five v37 audit recommendations and their v38 delivery.

2.2 The resilience layer: guard and backoff

The two resilience features are the ones whose value compounds silently. The database guard ends an entire class of setup failure: any fresh clone, sandbox or machine that boots the project through its own scripts now self-heals instead of serving an empty shell and failing fifteen checks into the bargain. The backoff layer is equally structural. Before v38, a 429 from a pool engine either failed the request immediately or relied on fixed sleeps; now a pure utility parses Retry-After headers (both the seconds form and the HTTP-date form, capped), computes exponential backoff with plus-or-minus 30 percent jitter, tries one bounded in-request retry when the wait is short enough, and otherwise parks the engine in a cooldown and fails over to the next route in the chain - with the keyed path getting the same treatment. Every AI feature in the product inherits this protection from one module, which is exactly how infrastructure should be bought: once, and for everything.

2.3 The fieldwork features: print and bridge

The two fieldwork features change what the Test of Control section *is*. The print engine means the questionnaire is no longer trapped in the browser: a blank copy (A4, tick-boxes, notes column, signature strip, full Arabic RTL when the app language is Arabic) can go to site as a PDF, and the completed run prints as a working paper with the verdict banner, weighted component scores and every marked answer. The bridge then closes the loop this product has been building for three versions: an auditor fills the ICQ, gets a **STRONG** verdict, clicks one button, and lands in the audit program customizer with the industry already selected and the live verdict plus top gaps written into the concerns - the questionnaire stops being a destination and becomes the on-ramp to the engagement itself. The implementation is deliberately conservative (a transient store field and an effect-free dialog-open pattern) which is why it survived the strict React hooks lint rule without hacks.

2.4 IFRS 16 at the flagship bar

The IFRS 16 rewrite applies the depth bar IFRS 15 established in v36, and the measuring stick is worth stating precisely because block counts alone flatter shallow content. The rebuilt standard carries **98 blocks and 5,859 English words against 5,111 Arabic words - an 87 percent parity ratio**. It embeds **460 numeric figures**, and its 13 journal blocks carry 38 rows of which 33 (87 percent) show actual amounts - the measure that separates a teaching journal from a decorative one. One running case (Delta Company: a four-year office-floor lease at 50,000 per year in arrears, incremental borrowing rate 6 percent) threads through the entire lessee engine - initial measurement, year-two interest and depreciation with the current/non-current split, CPI remeasurement, a term-extension modification - and then mirrors into the lessor side, net investment schedule, and a failed-sale sale-and-leaseback with numbers. The exam-method close (a four-line engine formula and a five-move method) matches how the DipIFR papers actually test this standard. The reference library now has two flagships at 110 and 98 blocks against a floor of 45 - the unevenness that remains is a deliberate triage decision, not an oversight.

2.5 What changed in the numbers

Metric	v37 audit	This audit
App version	37.0.0	38.0.0
Lines of TypeScript under test	75,783	76,662 (+879)
Test suites / automated checks	21 / 2,117	22 / 2,204 (+87)
Browser end-to-end checks	19	18 (re-focused on v38 paths)
IFRS revision blocks	2,125	2,160 (+35)
IFRS flagship standards	1 (IFRS 15 at 110)	2 (+ IFRS 16 at 98)
Database environment	manual restore	auto-restore guard
AI failure handling	fixed sleeps	backoff + cooldown + failover
Fieldwork exports	markdown only	PDF print: blank ICQ + verdict
Questionnaire-to-program flow	none	one-click bridge with prefill
Commits on main	31	33

Metric	v37 audit	This audit
CI pipeline	failing (unnoticed)	failing (now diagnosed)

Table 4 - Movement between the two audits (October 2026).

3. Product Inventory - What Exists Today

3.1 The platform at a glance

The platform stands at 76,662 lines of TypeScript across 50 API routes, 54 audit-domain React components and roughly two dozen top-level views, organized as a single Next.js application with Prisma over SQLite in development and a snapshot-provisioned database on Vercel. The architecture remains deliberately boring: one store, typed API routes with runtime guards on every AI endpoint, and content libraries that live in typed source modules rather than the database where generation is not needed. The bilingual requirement is enforced at the type level - every user-facing string is a {en, ar} pair - which is why the Arabic parity numbers throughout this report are structural facts rather than aspirations.

3.2 The learning spine

The learning spine holds 41 courses: 22 in the Arabic Academy (the largest single family, verified by a dedicated script and by the live RTL browser pass), 14 on the international standards, two IFRS courses, and one each for the Egyptian framework, analytics and open curated content. The courses carry 990 lessons and 27 quizzes, with 146 supplementary materials in the library. The engagement machinery around them - spaced repetition scheduling, XP levels, streaks, achievements, certificates - was verified by its own battery suites in this run, as it has been since v12 and v20.

3.3 Exam preparation

Exam preparation rests on a 2,685-question bank with 2,330 questions carrying Arabic stems (87 percent). The distribution is accounting-heavy by design - 1,668 accounting, 640 auditing, 274 ethics, 103 on Egyptian regulation - mirroring the DipIFR syllabus weighting, with difficulty spread across 483 easy, 1,653 standard and 549 hard items. The paper archive serves 27 real DipIFR exams from 2013 through 2025 (34 MB, self-hosted so no external link can rot), opened inside the site since v35, with an AI question generator and marker for the current formats. The bank API answered this campaign's production probe with the full live census, proving the deployed database is the complete one.

3.4 The reference library

The reference library now splits into two tiers by design. The floor tier covers all 41 IFRS standards at 45 to 52 blocks each - definitions, cores, trees, examples, tips, the structure a learner needs to orient. The flagship tier is where depth lives: IFRS 15 Revenue at 110 blocks and IFRS 16 Leases at 98, both carrying running numeric cases and journal sets that approach the depth of the printed revision notes. Alongside them, the Test of Control

library covers 45 industries across 10 sectors with 496 questions (22 core plus 474 industry-specific, 10 to 13 per industry). The interface dictionary holds 1,064 bilingual string pairs across 40 sections.

3.5 AI tooling

The AI tooling runs on a six-route keyless pool with a workspace engine first (serving GLM answers directly), fallbacks across public endpoints, and a keyed path for vision; the model registry exposes six selectable models with routing rules that the battery verifies against live calls. v38 added the backoff-and-cooldown layer beneath all of it. The feature surface spans the tutor chat with conversation history, the industry analyzer, the KAM drafter, podcast generation with edge TTS voices, exam question generation and marking, the Test of Control generator, and the program customizer that the new bridge feeds.

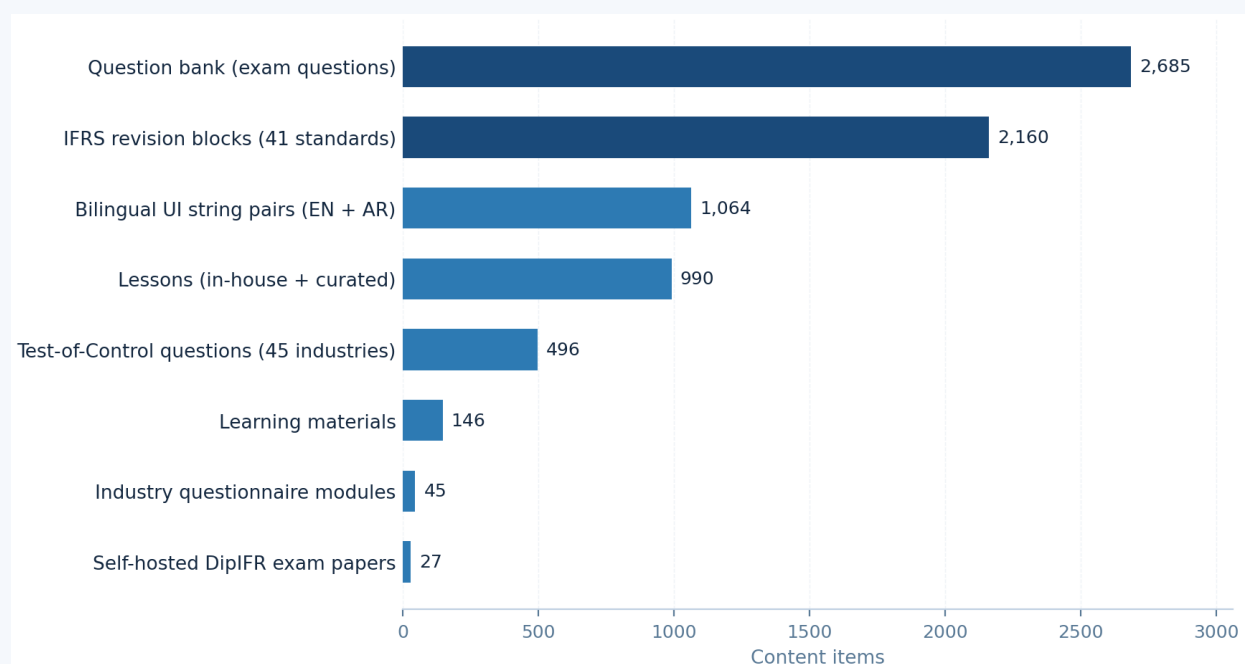


Figure 2 - Content assets by module (live database census and content libraries, October 2026).

Asset	Count
Courses (Arabic Academy / standards / other)	41 (22 / 14 / 5)
Lessons	990
Quizzes	27
Bank questions (with Arabic stems)	2,685 (2,330)
Learning materials	146
IFRS standards / revision blocks	41 / 2,160
IFRS flagship standards (blocks)	IFRS 15: 110 · IFRS 16: 98
ToC industries / sectors / questions	45 / 10 / 496

Asset	Count
DipIFR exam papers self-hosted	27 (34 MB)
Bilingual UI string pairs	1,064
API routes / audit components	50 / 54

Table 5 - Content inventory: database and libraries.

4. Health Assessment - Where We Stand

4.1 Strengths

The project's defining strength remains its verification culture, and v38 deepened it. Every release carries its own regression suite forward - twenty-two now - and the suites are not ceremonial: this campaign watched them catch a real environment defect (the empty database), verify live AI calls including vision routing, and pin deployment-level facts like the service-worker stamp matching the package version. The release process itself has settled into a rhythm where the audit's five recommendations became one shipped release with 87 new checks proving them - the loop from assessment to action to evidence is short and it is closing faster each cycle.

The second strength is structural restraint. Zero TODO markers in the tree; one boring stack doing the work; resilience bought as small pure modules (a backoff calculator, a DB probe, a print engine) rather than frameworks; content depth measured in words, figures and journal rows rather than headline counts; and bilingualism enforced by the type system so Arabic parity is never a retrofit. The new resilience layer also removed the project's dependence on luck: the two failure modes this codebase actually experienced (thin environment, rate-limited pool) both now have engineered responses rather than runbook entries.

4.2 Risk register

The risk register below updates the v37 register: two risks closed by v38 (manual environment bootstrap, unhandled AI rate-limiting), one new high-severity finding from this campaign (the CI pipeline), and three carried forward with their status honestly marked.

Risk	Severity	Impact	Mitigation
CI pipeline red for 23+ consecutive runs	HIGH (new)	Every push since 27 Sept failed CI; deployments bypass it via Vercel, so the 2,204-check moat gates nothing in practice. Root causes: the CI database is an empty scratch SQLite (the v20 suite fails 15 zero-count checks - the exact failure v38's ensure-db guard fixes locally but CI never calls), the latest two runs were never assigned a hosted runner and were cancelled after 15 minutes, and the workflow's branch filter is corrupted ('ain'] for 'main').	Wire ensure-db into the workflow before the battery, fix the filter, pin ubuntu-24.04, make the check required. Hours of work, converts the moat into a gate.
Production user data is ephemeral	MEDIUM (carried)	The demo deployment runs the snapshot-fallback mode: the content database is gunzipped from the bundle into the serverless instance's temp dir, and writes (progress, notes, AI conversations) reset when the instance recycles. The export/import path exists as the manual mitigation.	Wire a managed Postgres (the code path is already implemented and documented) when real-user persistence matters more than zero-config demo purity.
Single-user auth	MEDIUM (carried)	The app runs as one workspace user with admin rights. Fine for a personal workspace; blocks teams, cohorts and any instructor-learner relationship the Academy name implies.	Multi-user accounts with roles is the strategic unlock; see chapter 5.
IFRS depth unevenness beyond the flagships	MEDIUM (reduced)	39 of 41 standards sit at 45-52 blocks against 98-110 for the two flagships; five standards remain below 48. A deliberate triage, but the gap is now visible in the product.	One flagship per release (IFRS 9 next by exam weight) until the high-weight standards are all deep.
AI pool is third-party infrastructure	LOW-MEDIUM (reduced)	The keyless pool has no SLA and its engines rotate. v38's backoff, cooldowns and failover removed the sharp edges - this campaign's live calls ran clean with zero 429s - but the dependency itself remains.	Keep the keyed path maintained as the escape hatch; monitor pool health in the models suite.
No production telemetry loop	MEDIUM (new)	Analytics APIs and an engagement dashboard exist in-product, but there is no evidence of production usage being reviewed - the product is verified by its builders, not yet observed through its users.	Once durable production data exists, review real usage monthly; let it drive the content roadmap.

Table 6 - Risk register, ordered by severity. Two v37 risks closed (environment bootstrap, unhandled rate-limiting); one new HIGH.

4.3 The CI blind spot

The CI finding deserves its own paragraph because it is the campaign's lesson about blind spots. The project tests everything except the thing that runs the tests. Twenty-three consecutive red builds produced no alert, no blocked deploy, and no visible consequence - Vercel deploys straight from the push, so a green deployment and a

red CI run coexist happily. The failure is doubly ironic: the most recent long runs show the battery *starting to run* in CI (the v20 suite reaches its checks before dying), and the fifteen failures it hits are the precise zero-count failures v38's ensure-db guard was written to eliminate - the fix exists, is deployed, and is simply not called by the one place that needs it. The runner-acquisition failures on the latest two runs are a separate infrastructure problem (account-level runner quota or capacity), which pinning an explicit runner image and retry policy will also address. Repairing this is the cheapest trust upgrade available: a green checkmark on every future push, earned by the same 2,204 checks this audit ran by hand.

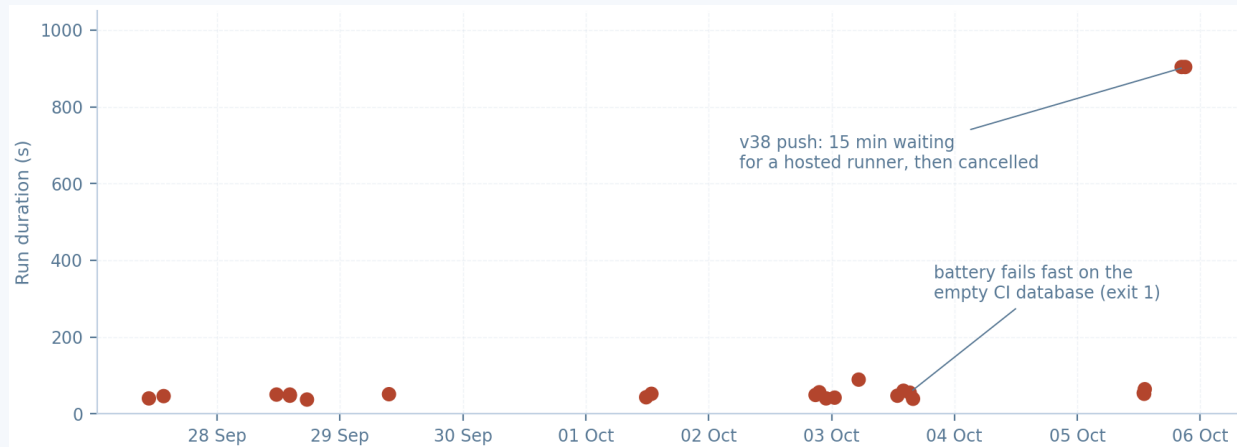


Figure 3 - GitHub Actions history: every run since 27 September 2026 failed. Fast failures are the battery dying on the empty CI database; the two 15-minute runs never received a hosted runner.

5. What We Can Do - The Roadmap

5.1 Immediate: repair the verification machinery

The immediate list is dominated by the CI repair because it converts every other investment into a protected one.

Repair CI (the audit's recommendation no. 1). Add bun scripts/ensure-db.ts (or the snapshot restore) before bun run test in .github/workflows/ci.yml; fix the corrupted 'branches: ain]' filters to 'main'; pin runs-on to ubuntu-24.04 so the label migration of October 19 cannot surprise; add a concurrency block with cancel-in-progress. Then make CI a required check so a red build blocks the merge, not just the conscience.

Adopt a performance budget. 76,662 lines on a client-heavy SPA deserves a measured first load: one Lighthouse pass, a bundle-size budget in CI (the same workflow can fail on regressions), and the service-worker precache list reviewed against what field users actually need offline.

Telemetry first cut. Start reviewing what the existing analytics routes already capture - even a monthly export of study sessions, exam attempts and AI feature usage from a durable copy would replace guessing with observation.

5.2 The next one to two releases

The next one to two releases compound what v37 and v38 built instead of opening new fronts.

Third IFRS flagship: IFRS 9. The depth-bar pattern is now twice-proven. IFRS 9 Financial Instruments is the natural next target by exam weight and difficulty perception; a 90-plus-block rewrite with one running case through classification, ECL staging and impairment would make the big three (15, 16, 9) all flagship.

Generalize the print engine. `icq-print.ts` proved the hidden-iframe pattern; the audit program, findings list and closeout summary are the natural next documents. One engine, four fieldwork artifacts, zero new dependencies.

The guided engagement flow. The pieces now exist end to end - questionnaire, verdict, bridge, customizer, program, findings, signoffs, closeout. A wizard that walks a first-time user through one complete engagement would turn a toolkit into a product narrative.

Verdict history per entity. Carried from the v37 recommendations and still the cheapest Test-of-Control depth win: past assessments with deltas answers the auditor's actual question - is the control environment improving?

5.3 The strategic horizon

The strategic horizon is about who the product serves and how it survives real use, not about more content.

Multi-user workspaces. Accounts, roles, shared engagements: the unlock that turns the Academy from a personal workspace into something a firm or a class can adopt. Everything in the current single-user model moves to an owner-scoped one.

Durable production data. Wire the already-implemented managed-Postgres mode for the real deployment. This is the prerequisite for the telemetry loop, for verdict history that survives recycles, and for any future account system.

Offline-first fieldwork. The service worker already precaches the shell; the field use case (ICQ at a client site with no signal) wants the questionnaire answers queued offline and synced later.

Content completion arc. Beyond the flagships: lift the five standards still under 48 blocks, then extend the depth bar down the exam-weight ranking (IAS 12, IAS 19, IFRS 3) until the floor tier and flagship tier meet.

5.4 Recommended scope for v39

The recommended v39 is therefore concrete: **CI repaired and required** (the trust fix), **IFRS 9 to the flagship bar** (the content fix), and **the print engine generalized to the audit program** (the fieldwork fix) - roughly one infrastructure day, one authoring pass and one feature day. The guided engagement flow then anchors v40, with the strategic items (workspaces, durable data, telemetry) sequenced behind them as the product's next era rather than its next release.

6. Release History and Reproducing the Audit

6.1 Version timeline v20 to v38

Nineteen shipped versions in the tracked window, each carrying its own regression suite forward. The biography the table tells: an exam engine first, content depth second, distribution and reliability third, fieldwork tooling now - and v38 as the release that turned an audit's recommendations into shipped, tested product.

Version	Highlight
v20	Exam engine: blueprint, SRS, ISA spine, Arabic depth pass
v21	IFRS 18 and Egypt regulation Arabic parity
v22	Past-paper question bank (366-check suite)
v23	ACCA deep past papers
v24	CFA / CMA / CPA skills bank
v25	Template-driven question generator families
v26	Flagship pools and generated podcast courses
v27	Podcast studio and bilingual episodes
v28	Edge TTS voices verified and wired
v29	Engagement tuning pass
v30	Shareable deep links and IFRS depth wave one
v31	IFRS Summaries beyond 750 blocks
v32	All 41 standards lifted past 45 blocks
v33	Service-worker self-heal for stale caches
v34	The real DipIFR past papers 2013-2025
v35	Sameh Zidan exams open inside the website
v36	IFRS 15 rewritten to the notes-PDF depth (110 blocks)
v37	Test of Control: 45 industries, verdict engine, AI generator
v38	Resilience + depth: DB guard, ICQ print, AI backoff, IFRS 16 flagship, program bridge

Table 7 - Release history, v20 through v38 (33 commits on main).

6.2 Reproducing this audit

This audit is fully reproducible from a clean clone in about twenty minutes of wall-clock time, and v38's guard removes the one step that used to be manual:

```
# 1. boot (the ensure-db guard self-heals the database)
bun install && bun run dev # predev runs scripts/ensure-db.ts

# 2. static gates
bunx tsc --noEmit && bunx eslint .

# 3. full regression battery - 22 suites, 2,204 checks
bun run test

# 4. end-to-end walkthrough of the v38 feature set
bash scripts/e2e-v38.sh

# 5. production build
bun run build
```

The campaign's conclusion in one sentence: v38 is the most verified state the product has ever reached - every local layer green, the deployment provably current, and the five audit recommendations not merely shipped but proven - and the one red light it found is in the machinery that should be proving it automatically, which makes repairing that machinery the first move of v39, not the last regret of v38.